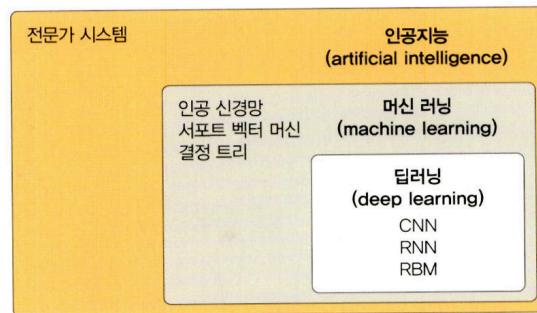


Week1 예습과제

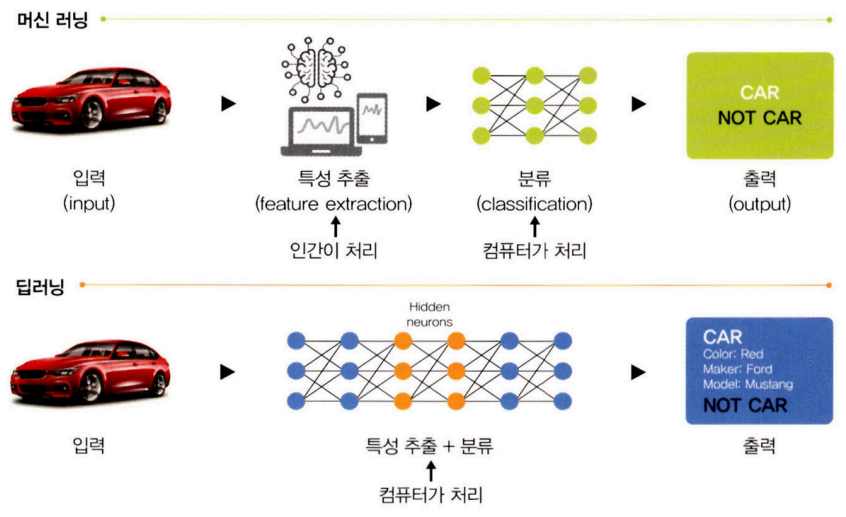
1.1 인공지능, 머신 러닝과 딥러닝

- 머신러닝과 딥러닝의 관계

▼ 그림 1-1 인공지능과 머신 러닝, 딥러닝의 관계



- 머신 러닝 vs 딥러닝
 - 머신 러닝: 주어진 데이터를 인간이 전처리 함(데이터의 특징을 스스로 추출하지 못함)
 - 딥러닝: 인간이 하던 작업을 생략해 컴퓨터가 스스로 분석 후 답을 찾음



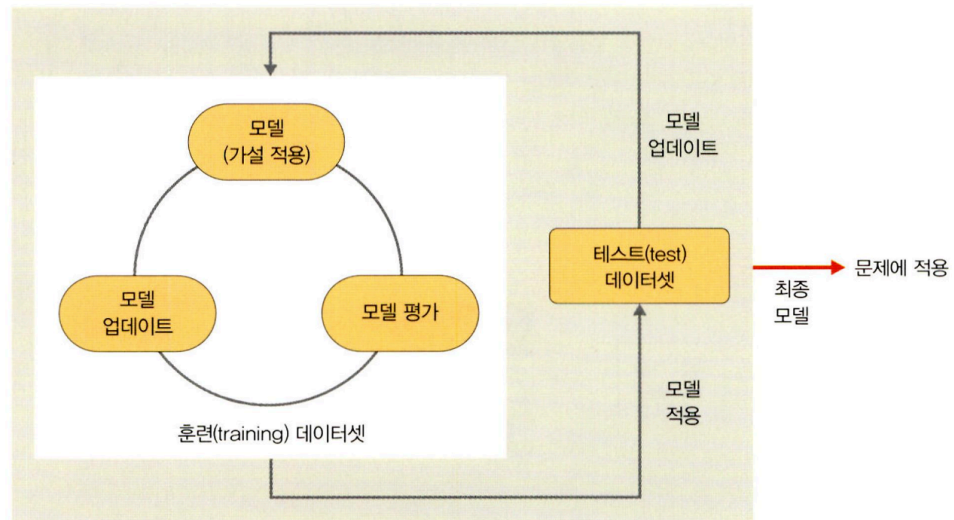
구분	머신 러닝	딥러닝
동작 원리	입력 데이터에 알고리즘을 적용하여 예측을 수행한다.	정보를 전달하는 신경망을 사용하여 데이터 특징 및 관계를 해석한다.
재사용	입력 데이터를 분석하기 위해 다양한 알고리즘을 사용하며, 동일한 유형의 데이터 분석을 위한 재사용은 불가능하다.	구현된 알고리즘은 동일한 유형의 데이터를 분석하는 데 재사용된다.
데이터	일반적으로 수천 개의 데이터가 필요하다.	수백만 개 이상의 데이터가 필요하다.
훈련 시간	단시간	장시간
결과	일반적으로 점수 또는 분류 등 숫자 값	출력은 점수, 텍스트, 소리 등 어떤 것이든 가능

1.2 머신 러닝이란

- 정의: 인공지능의 한 분야, 컴퓨터 스스로 대용량 데이터에서 지식이나 패턴을 찾아 학습하고 예측 수행

1.2.1 머신 러닝 학습 과정

- 학습 단계 / 예측 단계
 - 학습 단계: 훈련 데이터를 머신 러닝 알고리즘에 적용, 학습시켜 모형 생성
 - 예측 단계: 학습 단계에서 생성된 모형에 새로운 데이터를 적용하여 결과 예측
- 주요 구성 요소
 - 데이터
 - 학습 모델을 만드는 데 사용
 - 실제 데이터의 특징이 잘 반영되고 편향되지 않은 훈련 데이터를 확보하는 것이 중요
 - 데이터 수집 이후 훈련/테스트 데이터셋으로 분리해 사용(보통 8:2 비율)
 - 모델(가설): 머신 러닝의 학습 단계에서 얻은 최종 결과물
 - 아래와 같은 학습 절차를 반복하며 주어진 문제를 가장 잘 풀 수 있는 모델을 찾음



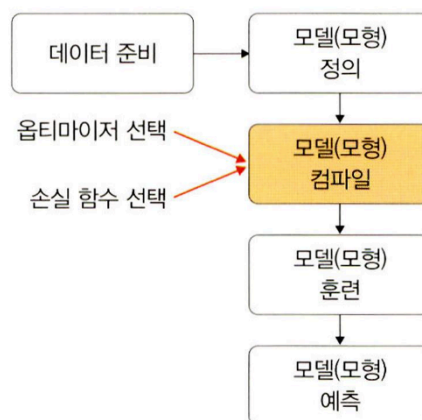
1.2.2 머신 러닝 학습 알고리즘

- 지도 / 비지도 / 강화 학습
 - 지도 학습: 정답이 무엇인지 컴퓨터에 알려 주고 학습 시킴
 - 비지도 학습: 정답을 알려주지 않고 비슷한 데이터를 범주화(클러스터링)하여 예측
 - 강화 학습: 게임을 하듯 보상이 커지는 행동은 자주, 줄어드는 행동은 덜 하도록 하여 학습 진행

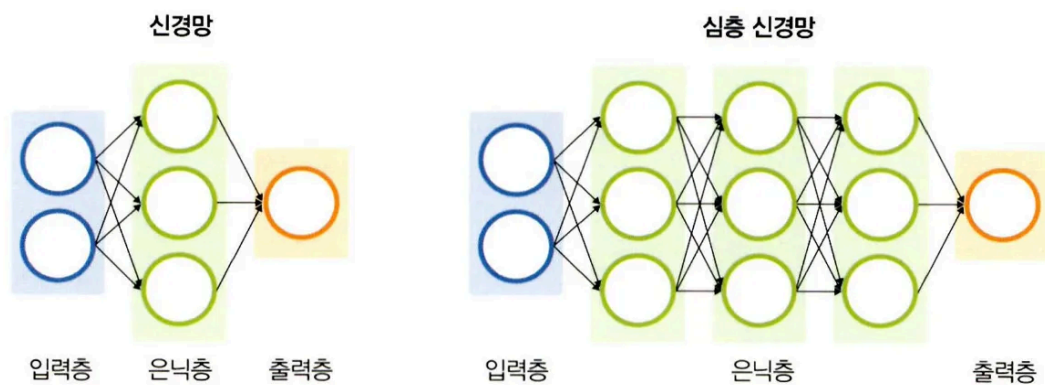
1.3 딥러닝이란

- 정의: 인간의 신경망 원리를 모방한 심층 신경망 이론을 기반으로 고안된 머신 러닝 방법의 일종

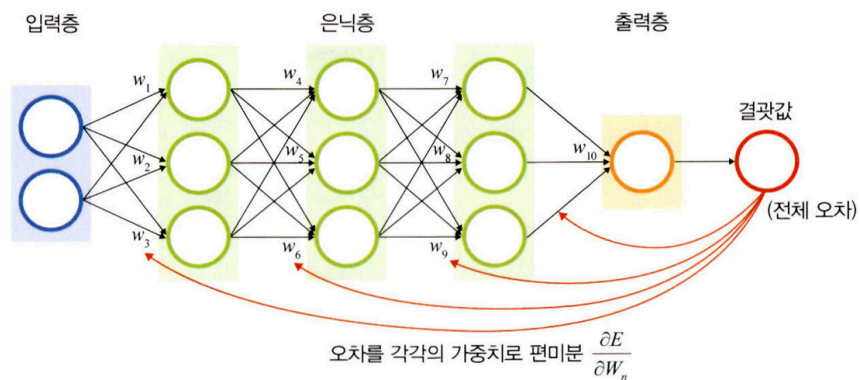
1.3.1 딥러닝 학습 과정



- 데이터 준비: 파이토치나 케라스에서 제공되는 데이터셋 사용, 캐글 등 공개된 데이터 사용
- 모델(모형) 정의: 신경망 생성, 은닉층 개수에 따른 성능과 과적합은 상충 관계에 있어 제대로된 신경망 생성이 중요
- 모델(모형) 컴파일: 활성화 함수, 손실 함수, 옵티마이저 선택
- 모델(모형) 훈련: 한 번에 처리할 데이터양 지정(배치와 에포크 선택)
- 모델(모형) 예측: 검증 데이터셋을 생성한 모델(모형)에 적용하여 실제 예측 진행
- 신경망과 역전파
 - 신경망: 심층 신경망을 사용해 머신 러닝과 달리 데이터셋의 어떤 특성들이 중요한 지 스스로에게 가르침



- 역전파: 가중치 값을 업데이트



*파이토치와 같은 프레임 워크를 이용하면 역전파 알고리즘을 자동으로 처리해주기 때문에 딥러닝 알고리즘 구현이 간단해짐

1.3.2 딥러닝 학습 알고리즘

- 지도 학습
 - 합성곱 신경망(CNN) - 컴퓨터 비전에서 주로 사용
 - 이미지 분류 - 데이터를 유사한 것끼리 분류할 때 사용
 - 이미지 인식 - 사진을 분석해 사물의 종류 인식
 - 이미지 분할 - 사물이나 배경 등 객체 간 영역을 픽셀 단위로 구분
 - 순환 신경망(RNN) - 시계열 데이터 분류 시 사용
 - 단점: 역전파 과정에서 기울기 소멸 문제 발생
 - LSTM
 - 순환 신경망의 문제점을 개선하고자 게이트 3개 추가
 - 망각 게이트, 입력 게이트, 출력 게이트를 도입해 기울기 소멸 문제 해결
- 비지도 학습
 - 워크 임베딩
 - 자연어를 컴퓨터가 이해하고 처리할 수 있도록 벡터로 변환
 - 단어의 의미를 벡터화하는 워드투벡터와 글로브를 가장 많이 사용
 - 자연어 처리 분야의 일종
 - 군집
 - 아무런 정보가 없는 상태에서 데이터를 분류
 - 한 클러스터 안의 데이터는 비슷하게, 다른 클러스터의 데이터와 구분되게 나눔
 - 머신 러닝 군집과 다르지 않음, 함께 사용하면 좋음
 - 전이 학습
 - 사전에 학습이 완료된 모델을 원하는 학습에 맞춰 미세 조정
 - 학습이 완료된 모델, 완료된 모델을 활용하는 접근 방법 필요
 - 사전 학습 모델
 - 풀고자 하는 문제와 비슷하면서 많은 데이터로 이미 학습이 되어 있는 모델
 - 오랜 시간과 연산량 필요

- VGG, 인셉션, MobileNet과 같은 사전 학습 모델을 사용해 효율적인 학습 진행 가능

- 정리

구분	유형	알고리즘
지도 학습(supervised learning)	이미지 분류	• CNN • AlexNet • ResNet
	시계열 데이터 분석	• RNN • LSTM
비지도 학습(unsupervised learning)	군집(clustering)	• 가우시안 혼합 모델(Gaussian Mixture Model, GMM) • 자기 조직화 지도(Self-Organizing Map, SOM)
	차원 축소	• 오토인코더(AutoEncoder) • 주성분 분석(PCA)
전이 학습(transfer learning)	전이 학습	• 버트(BERT) • MobileNetV2
강화 학습(reinforcement learning)	—	마르코프 결정 과정(MDP)

2.1 파이토치 개요

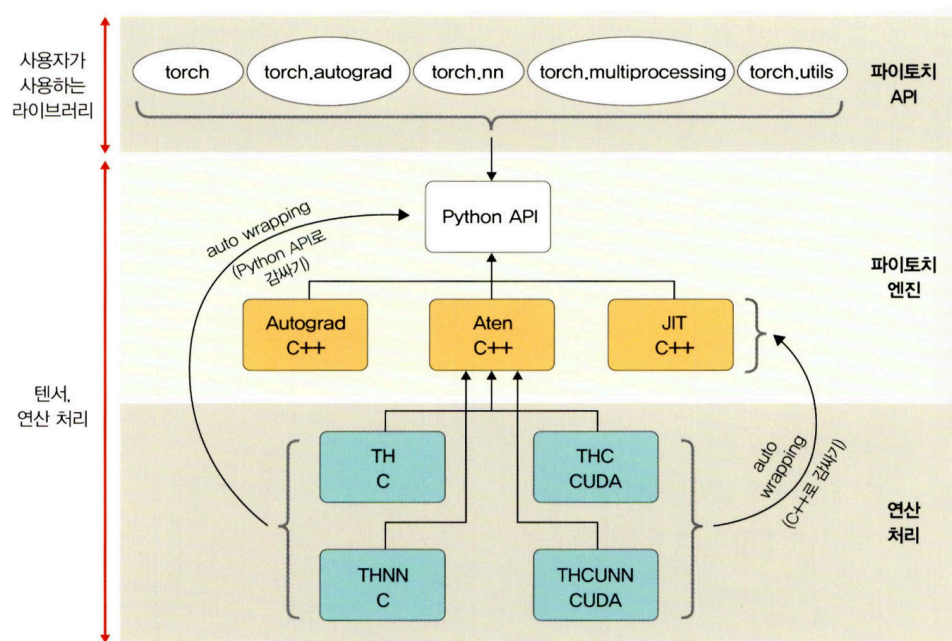
2.1.1 파이토치 특징 및 장점

- 특징: GPU에서 텐서 조작 및 동적 신경망 구축이 가능한 프레임워크
 - 용어 정리
 - GPU - 연산 속도를 빠르게 함
 - 텐서 - 파이토치의 데이터 형태로 단일 데이터 형식으로 된 자료들의 다차원 배열
 - 동적 신경망 - 훈련을 반복할 때마다 네트워크 변경이 가능한 신경망
- 장점
 - 효율적인 계산(단순함)

- 파이썬 환경과 쉽게 통합
- 직관적이고 간결한 디버깅
- 높은 성능
 - 낮은 CPU 활용
 - 빠르고 쉬운 학습 및 추론 속도
- 직관적인 인터페이스
 - 잦은 API 변경 없음

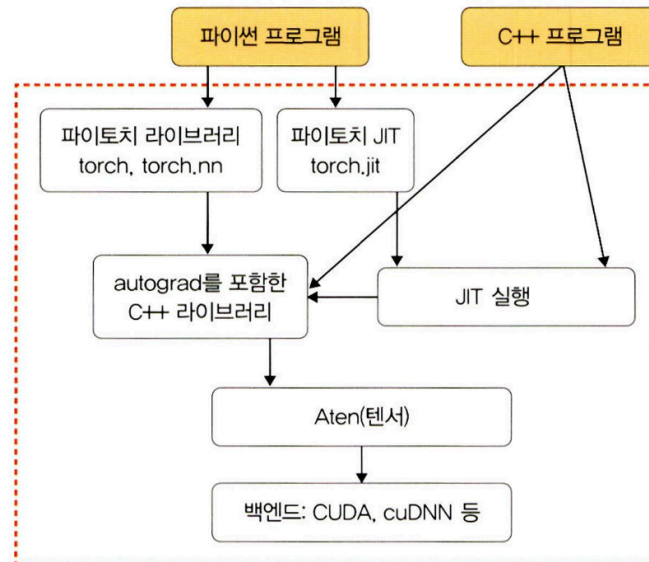
2.1.2 파이토치의 아키텍처

- 세 개의 계층



- 파이토치 API
 - 사용자가 이해하기 쉬운 API를 제공해 텐서에 대한 처리와 신경망 구축/훈련을 도움
 - 사용자 인터페이스를 제공하지만 실제 계산은 수행하지 않음(파이토치 엔진으로 계산 작업 전달)
 - 여러 패키지 제공
- 파이토치 엔진
 - 구성

- **Autograd C++** : 가중치/바이어스 업데이트 과정에서 필요한 미분 자동 계산
- **Aten C++** : 텐서 라이브러리 제공
- **JIT C++** : 계산 최적화를 위한 JIT 컴파일러
- **Python API**



C++로 감싼 다음 Python API 형태로 제공됨
→ 손쉬운 모델 구축, 텐서 사용

- 연산 처리
 - 상위의 API에서 할당된 거의 모든 계산 수행

2.2 파이토치 기초 문법

*파이토치에서는 텐서가 핵심!

2.2.1 텐서 다루기

- 텐서 생성 및 변환
 - 생성

```
import torch
print(torch.tensor([[1,2],[3,4]])) # 2차원 형태의 텐서 생성\
print(torch.tensor([[1,2],[3,4]], device="mps")) # GPU에 텐서 생성
```



```
print(torch.tensor([[1,2],[3,4]], dtype=torch.float64)) # dtype를 이용하여 텐서 생성
```

- ndarray로 변환

```
temp = torch.tensor([[1,2],[3,4]])  
print(temp.numpy()) # 텐서를 ndarray로 변환  
  
temp = torch.tensor([[1,2],[3,4]], device="mps")  
print(temp.to("cpu").numpy()) # GPU상의 텐서를 CPU의 텐서로 변환한 후 ndarray로 변환
```

- 텐서의 인덱스 조작

- 배열처럼 인덱스를 바로 지정하거나 슬라이스 등 사용 가능
- 텐서의 자료형

- `torch.FloatTensor` - 32비트의 부동 소수점
- `torch.DoubleTensor` - 64비트의 부동 소수점
- `torch.LongTensor` - 64비트의 부호가 있는 정수

```
temp = torch.FloatTensor([1, 2, 3, 4, 5, 6, 7]) # 1차원 벡터 생성  
print(temp[0], temp[1], temp[-1]) # 인덱스로 접근  
print('-----')  
print(temp[2:5], temp[4:-1]) # 슬라이스로 접근
```

- 텐서 연산 및 차원 조작

- 연산

- 다양한 수학 연산 가능, GPU를 사용해 빠르게 연산 가능
- 텐서 간 타입이 다르면 연산 불가능

```
v = torch.tensor([1, 2, 3])  
w = torch.tensor([3, 4, 6])  
print(w - v)
```

- 차원 조작

- 대표적으로 view 이용 - 넘파이의 reshape과 유사
- 이외 cat, transpose 등등 사용됨

```
temp = torch.tensor([
    [1, 2], [3, 4]
])

print(temp.shape)
print('-----')
print(temp.view(4,1)) # 2×2 행렬을 4×1로 변형
print('-----')
print(temp.view(-1)) # 2×2 행렬을 1차원 벡터로 변형
print('-----')
print(temp.view(1, -1)) # -1은 (1, ?)와 같은 의미로 다른 차원으로부터
                        # 해당 값을 유추하겠다는 뜻, 여기에서는 (1,4)
print('-----')
print(temp.view(-1, 1)) # (?, 1)의 의미, temp의 원소 개수(2×2=4)를
                        # 유지한 채 (?, 1)의 형태를 만족해야 하므로 (4, 1)
```

2.2.2 데이터 준비

*데이터가 이미지일 경우, 분산된 파일에서 데이터를 읽은 후 전처리 하고 배치 단위로 분할해 처리

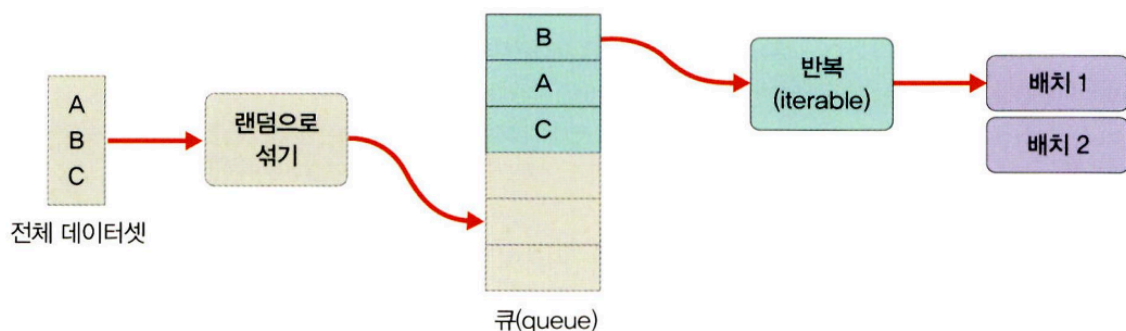
*데이터가 텍스트일 경우, 임베딩 과정을 거쳐 서로 다른 길이의 시퀀스를 배치 단위로 분할해 처리

- 단순히 파일을 불러와서 사용
 - 판다스 라이브러리를 통해 파일 불러옴
- 커스텀 데이터셋을 만들어서 사용
 - 데이터를 조금씩 나누어 불러 사용하는 커스텀 데이터셋을 사용
 - 데이터를 한 번에 메모리에 불러와 훈련시키는 것보다 시간과 비용의 측면에서 효율적
- 파이토치에서 제공하는 데이터셋 사용
 - 토치비전 - 파이토치에서 제공하는 데이터셋들이 모여 있는 패키지

2.2.3 모델 정의

모듈을 상속한 클래스를 사용

- 용어 정리(모델 vs 모듈)
 - 계층: 모듈 또는 모듈을 구성하는 한 개의 계층
 - 모듈: 한 개 이상의 계층이 모여서 구성된 것, 모듈이 모여 새로운 모듈을 만들 수 있음
 - 모델: 최종적으로 원하는 네트워크, 한 개의 모듈이 모델이 될 수 있음
⇒ 모델이 최종!
- 단순 신경망을 정의
 - `nn.Module`을 상속받지 않는 매우 단순한 모델을 만들 때 사용
 - 쉽고 단순한 구현 가능
- `nn.Module()`을 상속하여 정의
 - `__init__()`과 `forward()` 함수 포함
 - `__init__()`: 모델에서 사용될 모듈, 활성화 함수 등 정의
 - `forward()`: 모델에서 실행되어야 하는 연산 정의
- `Sequential` 신경망을 정의
 - `nn.Sequential`을 사용해 네트워크 모델을 정의하고, 가독성이 뛰어난 계산 코드 작성
 - `Sequential` 객체 안에 포함된 각 모듈을 순차적으로 실행
 - 계층이 복잡할 수록 효과가 뛰어남



학습에 사용될 데이터 전체를 보관하다, 모델 학습을 할 때 배치 크기만큼 데이터를 꺼내 사용
**주의: 데이터를 미리 잘라 놓는 것이 아니라 배치 크기만큼 데이터 반환하는 것!

- 함수로 신경망을 정의
 - `Sequential`을 이용하는 것과 동일

- 장점: 함수로 선언할 경우 저장해 놓은 계층들을 재사용 가능
- 단점: 모델이 복잡해짐

(복잡한 모델의 경우 함수보다 `nn.Module()`을 상속받아 사용하는 것이 편리)

2.2.4 모델의 파라미터 정의

- 손실함수
 - 학습하는 동안 출력값과 실제 값 사이의 오차 측정
 - 자주 사용되는 손실 함수
 - `BCELoss`
 - `CrossEntropyLoss`
 - `MSELoss`
- 옵티마이저
 - 데이터와 손실 함수를 바탕으로 모델의 업데이트 방법 결정
 - 옵티마이저의 주요 특성
 - `step()` 메서드를 통해 전달받은 파라미터 업데이트
 - 모델의 파라미터 별 다른 기준 적용 가능
 - `torch.optim.Optimizer(params, defaults)` → 모든 옵티마이저의 기본이 되는 클래스
 - `zero_grad()` 메서드를 통해 사용한 파라미터들의 기울기를 0으로 만들 수 있음
 - `torch.optim.lr_scheduler` → 에포크에 따라 학습률 조절
- 학습률 스케줄러
 - 지정한 횟수의 에포크를 지날 때마다 학습률 감소시킴
 - 학습 초기에는 빠른 학습을 진행하다 전역 최소점 근처에 다다르면 학습률을 줄여 최적점을 찾아 감
- 지표
 - 훈련과 테스트 단계 모니터링

2.2.5 모델 훈련

- 만들어 둔 데이터로 모델 학습
- $y = wx + b$ 라는 함수에서 적절한 w 와 b 의 값을 찾음

- 오차가 줄어들어 전역 최소점에 이를 때까지 파라미터 (w, b)를 계속 수정
- `optimizer.zero_grad()` 메서드를 호출해 기울기 초기화
→ `loss.backward()` 메서드를 이용해 기울기 자동 계산

2.2.6 모델 평가

- 주어진 테스트 데이터셋을 사용해 모델 평가

2.2.7 훈련 과정 모니터링

- 텐서보드를 이용해 학습에 사용되는 각종 파라미터 값의 변화 시각화, 성능 추적/평가

2.3 실습 환경 설정

2.4 파이토치 코드 맛보기